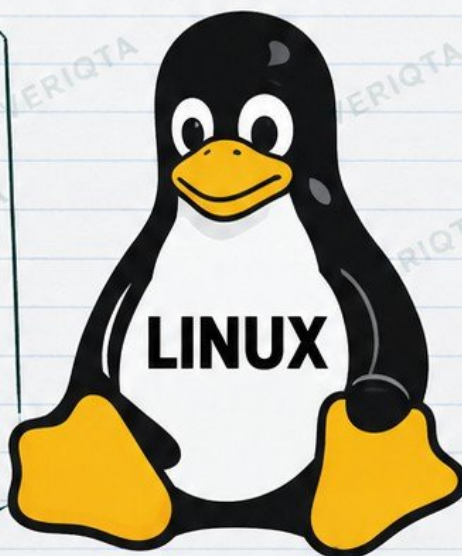


*Real Scenarios
Real Skills
Real Engineers*

50 SENIOR-LEVEL LINUX INTERVIEW SCENARIOS

REAL-WORLD PROBLEMS | PRACTICAL SOLUTIONS | SENIOR ENGINEER THINKING

- ✓ Production Issues
- ✓ Troubleshooting
- ✓ Performance
- ✓ Networking
- ✓ Security
- ✓ System Administration
- ✓ Containers
- ✓ Kernel & Internals
- ✓ Real Interview Questions



20 PAGES
50 SCENARIOS
SENIOR LEVEL

More Than
Commands
Think Like
A Senior Engineer



REAL
SCENARIOS



DETAILED
EXPLANATIONS



TROUBLESHOOTING
APPROACH



BEST
PRACTICES



INTERVIEW
READY

VERIQTA
DEVOPS | CLOUD | SRE | PLATFORM ENGINEERING

*Engineering
A Better You*

CPU SATURATION AND MISLEADING LOAD

Investigate | Analyze | Troubleshoot | Solve | Think Like a Senior Engineer

"High load does not always mean high CPU. Understand what the system is really doing."

1 Load average is 40, but CPU utilization is only 35%.
What do you investigate?

Key commands

- `uptime` # check load averages
- `top` # check process states (R, S, D)
- `vmstat 1 5` # check CPU, I/O wait, run queue
- `mpstat -P ALL 1 5` # per-core CPU usage
- `iostat -xz 1 5` # check disk I/O
- `pidstat -d 1 5` # per-process I/O stats

What this could mean

- Many processes are waiting (I/O, disk, network, locks)
- High run queue (R - runnable or D - uninterruptible sleep)
- CPU not the bottleneck, could be I/O, storage, or resource wait
- Check process states, I/O wait, and system saturation

What to look for

- High load average but high %wa (I/O wait)
- Many processes in D state (uninterruptible sleep)
- Disk or network latency
- NFS, database, or external dependency issues
- Contended locks or filesystem bottlenecks



Senior Engineer Tip

Load average = number of tasks in run queue + waiting for resources. It does not mean CPU is fully used.

2 One production service consumes 100% of a single core while the server appears healthy overall.
What do you investigate?

Key commands

- `top -H -p <PID>` # view threads
- `ps -eo pid,ppid,cmd,%cpu --sort=-%cpu`
- `pidstat -t -p <PID> 1 5` # per-thread CPU
- `perf top` # identify hot functions
- `strace -p <PID>` # see system calls (if needed)

Common causes

- Single-threaded application
- Hot code path or inefficient algorithm
- Missing concurrency (not using multiple threads)
- CPU-intensive background task (compression, encryption, etc.)
- No CPU affinity causing uneven load distribution

What to check

- Which thread is using CPU?
- Is it expected or a runaway process?
- Check recent changes or deployment
- Review code, logs, and configuration
- Determine if it can be multi-threaded or scaled horizontally

3 CPU utilization suddenly reaches 95% after a deployment.
How do you prove whether the application or operating system is responsible?

Key commands

- `top` # check %us, %sy, %ni, %id, %wa
- `mpstat -P ALL 1 5` # per-core breakdown
- `pidstat 1 5` # top processes by CPU
- `perf top` # kernel vs user space functions
- `sar -u 1 5` # historical CPU usage
- `dmesg -T | tail` # check kernel messages

Investigation approach

1. Confirm the timing (match with deployment time).
2. Identify top processes and threads.
3. Check if CPU is in user space (application) or kernel space (OS).
4. Look for kernel activity (interrupts, softirqs, kernel threads).
5. Compare before and after metrics.

How to interpret CPU breakdown

- High %us → application (user space)
- High %sy → kernel (OS)
- High %wa → I/O wait (check storage/network)
- High %si/%st → virtualized environment (steal time)
- Check if ksoftirqd, kworker, or interrupts are consuming CPU

Common root causes after deployment

- New code increasing CPU usage
- Missing cache or inefficient queries
- Increased traffic without scaling
- Misconfigured limits or resource contention?
- Background job running aggressively
- Kernel updates or changed system settings
- Increased logging or log processing
- Unexpected retry loops

Final checklist

- ✓ Identify the true cause (application or OS)
- ✓ Mitigate safely (scale, optimize, roll back, or fix config)
- ✓ Verify CPU returns to normal
- ✓ Implement monitoring and alerting to prevent recurrence

MEMORY PRESSURE AND OOM INCIDENTS

Understand | Diagnose | Solve | Prevent | Think Like a Senior Engineer

“Low free memory does not always mean the system is out of memory.”

1 A server has 32 GB RAM and only 500 MB appears free.
Is it actually running out of memory?

Key commands

- `free -h` # check memory usage
- `cat /proc/meminfo` # detailed memory info
- `vmstat 1 5` # memory, swap, system activity
- `top` # real-time process memory usage
- `ps -eo pid,comm,%mem,rss --sort=-%mem` # top memory consumers
- `smem -r | head` # detailed memory breakdown (if available)

What to understand

- Linux uses memory aggressively for page cache and buffers
- Low free memory is normal if available memory is healthy
- Check MemAvailable instead of MemFree
- Consider cache, buffers, and application memory (RSS)
- Look for swap usage and memory pressure indicators

What to check

- MemAvailable (not just MemFree)
- Page cache and buffers
- RSS and anonymous memory
- Swap usage
- Which processes are consuming memory
- Whether memory is actually under pressure



Senior Engineer Tip

Low free memory is expected. Focus on MemAvailable, swap usage, and whether the system is under memory pressure.

2 The Linux OOM killer terminates a critical application during peak traffic.
How do you investigate and prevent it?

Key commands

- `dmesg | grep -i oom` # check OOM killer logs
- `journalctl -k | grep -i oom` # kernel logs
- `cat /var/log/kern.log` # OOM events (depending on distro)
- `cat /proc/<pid>/oom_score` # check OOM score
- `cat /proc/<pid>/oom_score_adj` # check OOM score adjustment
- `cat /sys/fs/cgroup/memory/memory.limit_in_bytes` # cgroup limit (v1)
- `cat /sys/fs/cgroup/memory.max` # cgroup limit (v2)

Common causes

- Application using more memory than available
- Memory leaks
- Sudden traffic spike
- Multiple processes competing for memory
- Incorrect cgroup or container memory limits
- OOM killer selects the wrong process

Investigation steps

1. Confirm the OOM event and timestamp
2. Identify which process was killed and why
3. Check OOM score and memory usage of processes
4. Look for recent changes or traffic spikes
5. Review application logs and memory allocation patterns
6. Check container/cgroup memory limits if running in containers



Prevention strategies

- Optimize application memory usage
- Set appropriate cgroup/container memory limits
- Adjust oom_score_adj for critical services
- Monitor memory usage and set alerts before OOM occurs

3 Memory usage increases slowly for several days until the service crashes.
How do you identify and fix the issue?

Key commands

- `ps -eo pid,comm,%mem,rss --sort=-%mem` # top memory users
- `pmap -x <PID>` # memory map for a process
- `smem -r -p <PID>` # detailed memory breakdown
- `cat /proc/<PID>/status` # memory info for specific process
- `watch -n 60 free -h` # monitor memory over time
- `sar -r 1 10` # memory usage history (if sysstat installed)

What to look for

- Steady increase in RSS or anonymous memory
- Memory not released back to the OS
- Application memory leak (e.g. cache, list, connection objects)
- Growing number of processes or threads
- Large page cache usage due to file operations
- Container memory limits causing gradual OOM

Common tools for deeper analysis

- `valgrind` # detect memory leaks (dev/test)
- `heaptrack` # memory profiling (C/C++)
- `jemalloc / gperftools` # memory profiling (various languages)
- `VisualVM / JProfiler` # for Java applications
- `tracemalloc` # for Python applications

DISK I/O SATURATION

Analyze | Troubleshoot | Solve | Think Like a Senior Engineer

"High latency is often a storage problem, not a CPU or memory problem."

1 Application latency increases dramatically while CPU and memory remain healthy.

What do you investigate?

Key commands

- `iostat -xz 1 5` # detailed disk I/O stats
- `iotop -o` # real-time I/O by process
- `pidstat -d 1 5` # per-process I/O usage
- `vmstat 1 5` # check wa (I/O wait)
- `top` # confirm CPU and memory are healthy
- `dmesg | tail` # check filesystem usage
- `dmesg | tail` # check disk or file system errors

What this could mean

- Disk I/O saturation or high latency
- High IOPS with slow storage backend
- File system or block device bottleneck
- Application performing excessive disk operations
- I/O queue buildup causing request delays

What to check

- Disk I/O wait time (high %wa)
- Disk latency (await)
- IOPS and throughput
- Queue depth (avgqu-sz)
- Which processes are doing I/O
- File system vs block device issues
- Recent changes or deployments



Senior Engineer Tip

High application latency with low CPU usage often points to storage latency. Always check %wa and disk await before looking at the application.

2 Disk utilization reports 100%, but throughput appears low.

What do you investigate?

Key commands

- `iostat -x 1 5` # check %util, await, svctm, avgqu-sz
- `iotop -o` # see processes with high I/O
- `pidstat -d 1 5` # per-process read/write stats
- `lsblk -o NAME,ROTA,SIZE,TYPE` # check disk type (SSD/HDD)
- `cat /proc/diskstats` # raw disk stats
- `smartctl -a /dev/sdX` # check disk health (if available)

Common causes

- High I/O wait with small random I/O (high IOPS)
- Slow underlying storage (e.g. network storage)
- High queue depth causing requests to wait
- I/O scheduler or RAID/controller bottleneck
- File system metadata operations (e.g. many small files)
- Shared disk with noisy neighbour

Key metrics to analyze

- %util → how busy the device is (100% = saturated)
- await → average I/O wait time (high = slow storage)
- avgqu-sz → queue depth (high = requests waiting)
- r/s w/s → IOPS (reads/writes per second)
- kB/s MB/s → throughput (low with 100% util = small I/O)
- svctm → average service time

3 How do you identify if the issue is at the filesystem level or the block device level?

What steps would you take?

Additional commands

- `df -h` # filesystem usage
- `du -sh /path` # find large directories
- `ls -l /path | head` # check for many small files
- `find /path -type f | wc -l` # count files (inode usage)
- `cat /proc/mounts` # check mount options
- `iotop -o` # identify processes
- `pidstat -d 1 5` # per-process I/O
- `sar -d 1 5` # historical disk I/O stats

Filesystem vs Block Device Symptoms

- Filesystem: many small files, high inode operations, metadata-heavy workload, fsck errors, full filesystem
- Block device: high latency, high queue depth, hardware/storage backend issues, low throughput, controller or network storage problems

Investigation checklist

- ✓ Confirm the symptom (latency, errors, slow responses)
- ✓ Check disk metrics (iostat, sar, pidstat)
- ✓ Identify the top I/O consuming processes
- ✓ Determine if it is filesystem or block device
- ✓ Check disk health and recent changes
- ✓ Look for I/O patterns (random vs sequential, small vs large I/O)
- ✓ Verify storage backend (EBS, NFS, SAN, RAID, local disk)
- ✓ Implement fixes and monitor for improvement

FILESYSTEM FULL, BUT THE NUMBERS DO NOT MATCH

Investigate | Troubleshoot | Solve | Think Like a Senior Engineer

“Disk issues are common in production. Find the real cause, not just the symptom.”

1 **df reports 100% usage while du cannot account for the consumed space.**
What do you investigate?

Key commands

- `df -h` # check filesystem usage
- `du -sh /path` # check directory size
- `lsdf +L1` # find deleted files still open
- `ls -lah /proc/* /fd` # see open file descriptors
- `find / -xdev -type f -size +100M` # find large files
- `du -xh --max-depth=1 /` # check top level usage
- `mount | column -t` # check mount points

Common causes

- Deleted files still open by a process
- Large log files (rotated but still in use)
- Files in hidden directories
- Filesystem reserved blocks
- Snapshots or bind mounts
- Inode exhaustion (less common)

What to look for

- Deleted files with (deleted) in lsdf output
- Large log files still being written
- Differences between df and du due to open files
- Hidden directories (.cache, .local, application data)
- Docker/Container logs
- LVM snapshots or other mounts consuming space



Senior Engineer Tip

Use lsdf +L1 to find deleted files still held open by a process. Truncating the file or restarting the process can immediately free space without downtime.

2 **The filesystem has free storage capacity, but applications cannot create new files.**
What do you investigate?

Key commands

- `df -i` # check inode usage
- `ls -i | wc -l` # count files in a directory
- `mount | grep 'on /'` # check mount options (ro/rw)
- `dmesg | tail` # look for filesystem errors
- `quota -s` # check disk quotas (if enabled)

Common causes

- Inode exhaustion (no free inodes)
- Filesystem mounted read-only
- User or directory permissions
- Disk quota limitations
- Too many files in a single directory
- Filesystem corruption

What to check

- Inode usage (df -i)
- Mount options (read-only)
- Filesystem errors in dmesg
- Directory permissions and ownership
- Application-specific limits
- Large number of small files

3 **/var suddenly fills and production services begin failing.**
What do you investigate and fix?

Key commands

- `du -sh /var/*` # find largest directories
- `journalctl --disk-usage` # check journal size
- `ls -lah /var/log` # inspect log files
- `find /var -type f -size +100M` # find large files
- `docker system df` # check container disk usage
- `ncdu /var` # interactive disk usage viewer

Common causes

- Runaway logs (application, system, audit, Docker)
- Log rotation not working
- Core dumps or large temporary files
- Package manager cache (/var/cache)
- Container logs (/var/lib/docker)
- Monitoring/backup tools writing large data

Recovery steps

1. Identify the biggest directories and files.
2. Safely rotate or truncate logs (do not delete active logs).
3. Clean package cache if safe (e.g. apt clean).
4. Remove old containers/images (if applicable).
5. Verify services and monitor disk usage.
6. Implement log rotation and alerts to prevent recurrence.

FILE DESCRIPTOR EXHAUSTION

Diagnose | Troubleshoot | Solve | Think Like a Senior Engineer

“File descriptor limits can break production. Fix the cause, not just the limit.”

1

A high-traffic API suddenly reports “Too many open files.”
What do you investigate?

Key commands

- `ulimit -n` # check process limit
- `cat /proc/<pid>/limits` # view process limits
- `ls /proc/<pid>/fd | wc -l` # count open file descriptors
- `lsof -p <pid> | wc -l` # list open files
- `ss -s` # socket summary
- `ss -antp | awk '{print $1}' | sort | uniq -c` # connection states
- `netstat -an | grep <pid>` # view process sockets

What this could mean

- Process has reached its file descriptor limit
- Many open network connections (e.g. client sockets)
- File descriptor leak in the application
- Too many worker processes or threads
- High rate of short-lived connections
- System-wide limits may also be reached

What to check

- Current open file descriptors for the process
- Process and system limits
- Number of open sockets (ESTABLISHED, TIME_WAIT, etc.)
- Recent changes or traffic spike
- Whether the issue is limited to one process or system-wide
- Look for file descriptor leaks (files, sockets, pipes)



Senior Engineer Tip

Don't just increase the limit. Find out why file descriptors are not being closed. A leak will return and eventually exhaust the new limit.

2

Increasing ulimit -n temporarily fixes an incident, but the problem returns.
How do you find and fix the root cause?

Key commands

- `watch -n 5 'ls /proc/<pid>/fd | wc -l'` # monitor in real time
- `lsof -p <pid>` # see open files/sockets
- `ss -antp` # check connection states
- `cat /proc/sys/fs/file-max` # system-wide limit
- `sysctl fs.file-max=<value>` # (temporary) increase
- `cat /etc/security/limits.conf` # user limits
- `cat /etc/systemd/system.conf` # systemd limits

Common causes

- Application is not closing files or sockets properly
- Connection pool not releasing connections
- Long-lived connections or idle sockets
- File descriptor leak in a library or third-party component
- High number of threads or processes
- Insufficient system-wide limits

Investigation steps

1. Confirm the limit and current usage.
2. Identify what type of descriptors are open (files, sockets, etc.).
3. Check for steady growth over time.
4. Correlate with traffic patterns or recent deployments.
5. Review application logs and code for leaks.
6. Use profiling or tracing if needed (e.g. lsof, strace, eBPF tools).

3

How do you plan for file descriptor limits in a high-scale environment?
What best practices would you follow?

Useful configuration (examples)

- `ulimit -n 65536` # temporary (current shell)
- `/etc/security/limits.conf` # permanent user limits
- `/etc/systemd/system.conf` # system-wide defaults
- `fs.file-max = 2097152` # kernel limit
- `net.ipv4.tcp_fin_timeout = 30` # faster cleanup (if safe)

Best practices

- Set appropriate soft and hard limits for applications
- Configure system-wide limits (fs.file-max)
- Tune connection pools and timeouts
- Use keep-alives and close idle connections
- Monitor file descriptor usage and set alerts
- Load test to validate limits under peak traffic
- Fix leaks in the application code or dependencies

Monitoring signals

- ✓ Open file descriptors per process
- ✓ System-wide file descriptor usage
- ✓ Number of established and TIME_WAIT connections
- ✓ Trend over time (unexpected growth)
- ✓ Alerts before reaching 80% of limits

PROCESS STATES AND HUNG APPLICATIONS

Investigate | Troubleshoot | Solve | Think Like a Senior Engineer

"A running process does not always mean a healthy application."

1 A process cannot be killed even with kill -9.

What do you investigate?

Key commands

- `ps -eo pid,stat,comm,wchan` # check process state (look for D)
- `top` or `htop` # view overall system state
- `cat /proc/<pid>/stack` # see kernel stack (what it's waiting for)
- `cat /proc/<pid>/wchan` # check wait channel
- `lsof -p <pid>` # open files and resources
- `strace -p <pid>` # trace system calls (if possible)
- `dmesg | tail` # check kernel messages

What this could mean

- Process is in uninterruptible sleep (D state)
- Waiting on disk I/O, network, or filesystem
- Kernel is waiting for a resource (not a user-space issue)
- Stuck NFS, blocked storage, or hardware issue
- kill -9 cannot terminate a process in D state

What to check

- Confirm the process state (is it D?)
- Identify what resource it's waiting for
- Check disk, filesystem, or network issues
- Look for recent storage or NFS problems
- Review kernel logs for I/O errors
- Check if the issue is affecting other processes



Senior Engineer Tip

If a process is in D state, focus on the underlying resource (e.g. disk, NFS, network). Killing the process will not work.

2 Hundreds of zombie processes appear on a production host.

What do you investigate?

Key commands

- `ps -eo pid,ppid,stat,cmd | grep Z` # find zombie processes
- `ps tree -p | grep <parent>` # check parent process tree
- `top` # view process states
- `cat /proc/<ppid>/status` # check parent process info
- `strace -p <ppid>` # see if parent is waiting/reaping
- `systemctl status <service>` # if related to a service

Common causes

- Parent process not reaping child processes
- Application bug or improper signal handling
- Long-running parent process
- High process churn (e.g. web server, job workers)
- System under heavy load
- Sometimes caused by containers or PID 1 issues

What to check

- Identify the parent process (PPID)
- Verify why the parent is not reaping children (wait/waitpid)
- Check application logs and code behavior
- Look for process management issues in containers
- Ensure PID 1 in container is handling signals correctly
- Restart or fix the parent process (after confirming impact)

3 A service is running according to systemd but no longer serves traffic.

What do you investigate?

Key commands

- `systemctl status <service>` # check service status
- `journalctl -u <service> -n 100` # view recent logs
- `ss -tulnp | grep <port>` # check if port is listening
- `ps -ef | grep <service>` # verify running process
- `curl -v http://localhost:<port>` # test local connectivity
- `lsof -p <pid>` # check open files
- `cat <config file>` # review configuration

Common causes

- Application is running but unhealthy (not serving requests)
- Service started successfully but failed after initialization
- Dependency failure (DB, cache, external API)
- Port not listening or firewall blocking
- Misconfiguration (wrong environment, config file, or permissions)
- Service stuck or deadlocked

What to check

- Confirm the process is running and healthy
- Verify the service is listening on the expected port
- Check application logs for errors
- Test local and external connectivity
- Check dependencies (DB, cache, other services)
- Review recent changes or deployments
- Compare with a known good state

SYSTEMD PRODUCTION FAILURES

Investigate | Troubleshoot | Solve | Think Like a Senior Engineer

"A service working manually but failing under systemd usually indicates a difference in environment, permissions, or service configuration."

1

A service works when started manually but fails when started through systemd.

What do you investigate?

What this could mean

- Different environment variables
- Different user and group context
- Missing permissions or access to files
- Required dependencies not started
- Working directory or relative paths
- Service file misconfiguration
- systemd sandboxing restrictions (ProtectSystem, etc.)

Key commands

- `systemctl status <service>` # check service status
- `journalctl -u <service> -n 100` # view logs
- `systemctl cat <service>` # view unit file
- `systemctl show <service>` # show all unit properties
- `ps -ef | grep <service>` # check running processes
- `env | sort` # compare environment
- `sudo -u <user> env` # check user environment
- `ls -l /path/to/binary` # verify permissions

What to check

- Environment variables (Environment=, EnvironmentFile=)
- User and group (User=, Group=)
- WorkingDirectory
- ExecStart path and arguments
- File and directory permissions
- Required dependencies (After=, Wants=, Requires=)
- Difference in PATH and shell environment
- systemd sandboxing options (ProtectSystem, PrivateTmp, etc.)



Senior Engineer Tip

Compare the environment of a manually started service with the systemd service using: `systemctl show <service> | grep -E 'User|Group|Environment|WorkingDirectory'`.

2

A critical service enters a restart loop after reboot.

What do you investigate?

Common causes

- Service failing due to misconfiguration
- Missing dependencies (network, database, mount)
- Permission issues
- Service starting before required resources are available
- Crash loop due to application errors
- Incorrect restart policy
- Resource limits (memory, file descriptors)

Key commands

- `systemctl status <service>` # check status and recent logs
- `journalctl -u <service> -b` # view logs from current boot
- `systemctl show <service>` # show unit properties
- `systemctl list-dependencies <service>` # check dependencies
- `systemctl get-default` # check boot target
- `cat /etc/fstab` # verify required mounts
- `df -h` # check filesystem availability
- `dmesg | tail` # check kernel messages

What to check

- Error messages in journalctl
- Service dependencies and ordering
- Required mounts and network availability
- Restart policy (Restart=, RestartSec=)
- Application logs and exit codes
- Resource limits (memory, open files)
- Confirm the service is not failing due to timeout

Additional useful commands

- `systemctl edit <service>` # create an override file
- `loginctl show-user <user>` # check user session info
- `systemd-analyze blame` # check boot time delays
- `systemd-analyze critical-chain` # check dependency chain
- `systemctl list-units --failed` # list failed services
- `systemctl reset-failed <service>` # clear failed state

Prevention strategies

- Ensure all dependencies are correctly defined
- Use appropriate restart policies with limits
- Add health checks and proper logging
- Validate configuration before deployment
- Use systemd service hardening carefully (test before enabling)
- Monitor service failures and set up alerts
- Document environment and permission requirements

LINUX BOOT FAILURE

Diagnose | Troubleshoot | Solve | Think Like a Senior Engineer

“Boot problems are stressful, but with a structured approach, you can recover quickly.”

1 A production VM fails to boot after a kernel or configuration change.

What do you investigate?

Key commands

- lsblk # check block devices
- cat /proc/cmdline # view kernel parameters
- cat /etc/default/grub # check GRUB settings
- grub2-editenv list # view saved GRUB entries
- grub2-mkconfig -o /boot/grub2/grub.cfg # regenerate GRUB config
- journalctl -b -1 # view previous boot logs
- dmesg | tail # check kernel messages
- file /boot/vmlinuz-* # verify kernel files
- lsinitrd /boot/initramfs-*.img | less # inspect initramfs contents

What this could mean

- Broken kernel or incompatible module
- Incorrect GRUB configuration
- Initramfs not generated or corrupted
- Filesystem or mount configuration error
- Driver or hardware compatibility issue
- Failed systemd service during boot

What to check

- Can you access the GRUB menu?
- Is the correct kernel being loaded?
- Any kernel panic or error messages?
- Are the initramfs and kernel files present and valid?
- Recent changes to GRUB, kernel, or drivers?
- Is the root filesystem available and not corrupted?
- Any hardware or virtualization changes?
- Check previous boot logs: journalctl -b -1



Senior Engineer Tip

Always keep at least one known good kernel version.
Use the GRUB menu to boot an older kernel when in doubt.

2 A host enters emergency mode after reboot.

What do you investigate?

Key commands

- systemctl status # check service status
- journalctl -xb # view detailed boot logs
- cat /etc/fstab # check mount configuration
- lsblk -f # view block devices and filesystems
- blkid # check filesystem UUIDs
- fsck -n /dev/sdX # check filesystem (read-only)
- mount -a # test mounting all filesystems
- systemctl list-units --failed # list failed systemd units
- dmesg | grep -i error # check kernel error messages
- findmnt # view current mounts

What this could mean

- Filesystem mount failure (e.g. wrong UUID, missing device)
- Errors in /etc/fstab
- Disk or filesystem corruption
- LVM, RAID, or network storage not available
- systemd service failure blocking boot
- Incorrect SELinux/AppArmor configuration
- Broken dependency or missing critical file

What to check

- Which filesystem or mount is failing?
- Is the device/UUID in /etc/fstab correct?
- Is the disk available (local, LVM, RAID, network)?
- Are there filesystem errors (run fsck if needed)?
- Any required service failing before reaching multi-user target?
- Check SELinux/AppArmor denials.
- Review logs with journalctl -xb.

Recovery steps

1. Boot into GRUB and select an older kernel (if available).
2. Use rescue mode or live ISO if the system will not boot.
3. Fix /etc/fstab or filesystem issues.
4. Run fsck to repair filesystem errors.
5. Regenerate initramfs if corrupted.
6. Update GRUB configuration and reinstall if necessary.
7. Verify and test the fix, then reboot.

Prevention strategies

- Keep multiple kernel versions installed.
- Test configuration changes in a non-production environment.
- Use UUIDs instead of device names in /etc/fstab.
- Monitor disk health and filesystem errors.
- Keep backups and snapshots (especially for VMs).
- Document recovery procedures.
- Use configuration management and change control.

NETWORK CONNECTIVITY FAILURE

Diagnose | Troubleshoot | Solve | Think Like a Senior Engineer

“Network issues are rarely just the network. Follow the path, verify each layer, and find the real cause.”

1 The server can reach some networks but not others. What do you investigate?

Key commands

- `ip addr show` # check network interfaces
- `ip route show` # view routing table
- `ip route get <destination>` # see the route to a specific host
- `ping <ip>` # test basic connectivity
- `traceroute <ip>` # trace the path to a host
- `ss -tulnp` # check listening ports
- `nft list ruleset` # view firewall rules (nftables)
- `tcpdump -i <interface>` # capture and analyze packets
- `mtr <destination>` # continuous network diagnostics

What this could mean

- Incorrect routing or missing route
- Firewall blocking traffic
- Network ACL or security group rules
- Interface or link issues
- Asymmetric routing
- VLAN or subnet misconfiguration

What to check

- Is the correct route in place?
- Is the interface up and has an IP address?
- Are firewall rules blocking the traffic?
- Is there an issue with VLANs, subnets, or network ACLs?
- Does the problem occur across specific networks only?
- Are there asymmetric routing or NAT issues?
- Can you reach the destination from another host?



Senior Engineer Tip

Test from multiple points (same server, different server, different network) to identify where the connectivity breaks.

2 An application can connect using an IP address but fails when using the hostname. What do you investigate?

Key commands

- `dig <hostname>` # detailed DNS query
- `nslookup <hostname>` # simple DNS lookup
- `getent hosts <hostname>` # check system name resolution
- `resolvectl status` # view current DNS configuration
- `cat /etc/resolv.conf` # check resolver settings
- `ping <hostname>` # test hostname resolution
- `curl -v http://<hostname>` # test application connectivity

What this could mean

- DNS resolution issue
- Incorrect or missing DNS records
- Multiple DNS servers with different results
- Negative DNS caching
- Search domain or /etc/resolv.conf misconfiguration
- Split-horizon DNS (internal vs external)

What to check

- Does the hostname resolve to the correct IP?
- Are you using the expected DNS server?
- Is there a mismatch between internal and external DNS?
- Are there multiple A or CNAME records?
- Is there DNS caching (client or server) causing stale results?
- Does /etc/hosts have an incorrect entry?
- Can you resolve the hostname from other servers/clients?

3 Connections work locally but fail from remote clients. What do you investigate?

Key commands

- `ss -tulnp` # check listening addresses
- `ip addr show` # verify correct interface
- `nft list ruleset` # check firewall rules
- `telnet <ip> <port>` # test port connectivity
- `nc -vz <ip> <port>` # test with netcat
- `curl -v http://<ip>:<port>` # test HTTP/HTTPS connectivity
- `tcpdump -i any port <port>` # capture and analyze packets

What this could mean

- Service is bound to localhost (127.0.0.1) only
- Firewall blocking external access
- Incorrect security group or network ACL
- Application is listening on the wrong interface
- NAT or port forwarding issue
- Network path or ISP filtering

What to check

- Is the service listening on 0.0.0.0 or only 127.0.0.1?
- Are firewall rules allowing external traffic?
- Is the correct port open (and no port conflicts)?
- Is there a security group or network ACL blocking access?
- Does the application require specific source IPs?
- Can you reach the server from another network or device?
- Check application and firewall logs for dropped connections.

TCP CONNECTION INCIDENTS

Diagnose | Troubleshoot | Solve | Think Like a Senior Engineer

“Understand how TCP works, because most real-world outages involve connections.”

1 Thousands of connections accumulate in TIME_WAIT. What do you investigate?

Key commands

- `ss -tan state time-wait` # view TIME_WAIT connections
- `ss -s` # socket summary
- `netstat -ant | grep TIME_WAIT` # check TIME_WAIT count
- `cat /proc/sys/net/ipv4/ip_local_port_range` # check ephemeral port range
- `cat /proc/sys/net/ipv4/tcp_tw_reuse` # check port reuse setting
- `cat /proc/sys/net/ipv4/tcp_fin_timeout` # check TIME_WAIT timeout

What this could mean

- High volume of short-lived connections
- Application not reusing connections
- Normal TCP behavior, but expected to be high
- Ephemeral port exhaustion risk
- Load balancer or client-side issue
- Misconfigured keep-alive or connection pooling

What to check

- Is TIME_WAIT expected for your workload?
- Are connections being re-used (keep-alive, pooling)?
- Is the ephemeral port range sufficient?
- Is `tcp_tw_reuse` or `tcp_fin_timeout` tuned correctly?
- Any client behavior causing excessive new connections?
- Is a load balancer creating many short-lived connections?



Senior Engineer Tip

TIME_WAIT is usually normal, but if it consumes all ephemeral ports, new connections will fail.

2 Production connections remain stuck in SYN_SENT or SYN_RECV. What do you investigate?

Key commands

- `ss -tan state syn-sent` # view SYN_SENT connections
- `ss -tan state syn-recv` # view SYN_RECV connections
- `ss -lnt` # check listening sockets
- `netstat -s | grep -i syn` # check SYN-related counters
- `cat /proc/sys/net/ipv4/tcp_max_syn_backlog` # check SYN backlog size
- `tcpdump -ni <interface> tcp port <port>` # capture and analyze packets

What this could mean

- Server is not reachable or dropping SYN packets
- Firewall or security group blocking traffic
- Server is overloaded (SYN backlog full)
- Network latency or packet loss
- Incorrect routing or asymmetric routing
- DDoS or connection flood

What to check

- Is the service actually listening on the expected port?
- Are firewall rules allowing the traffic?
- Is the SYN backlog being exceeded?
- Are there network issues, packet loss, or high latency?
- Is the server under heavy load?
- Check for SYN floods or unusual traffic patterns.

3 Connections randomly reset during traffic spikes. What do you investigate?

Key commands

- `ss -tan state established` # view established connections
- `ss -tan state reset` # view reset connections
- `netstat -s | grep -i reset` # check TCP reset counters
- `dmesg | grep -i tcp` # check kernel TCP messages
- `sysctl net.ipv4.tcp_*` # review TCP settings
- `tcpdump -ni <interface> tcp` # capture and analyze resets

What this could mean

- Load balancer or firewall closing idle connections
- Application crashing or closing connections
- Network devices resetting connections
- Kernel resource exhaustion (memory, sockets, ports)
- Keep-alive timeouts or misconfiguration
- Middlebox interference (NAT, firewall, proxy)

What to check

- Are resets coming from the server, client, or network device?
- Check application and load balancer logs.
- Verify keep-alive and idle timeout settings.
- Look for resource exhaustion (memory, file descriptors, ports).
- Analyze packet captures to identify who is sending the RST.
- Confirm there are no security devices interfering.

DNS FAILURE UNDER PRODUCTION LOAD

Diagnose | Troubleshoot | Solve | Think Like a Senior Engineer

"DNS problems can bring down healthy systems. Understand the full resolver path."

1 Applications intermittently report DNS timeouts while network connectivity remains healthy. What do you investigate?

Key commands

- `dig <domain>` # query DNS server directly
- `nslookup <domain>` # simple DNS query
- `resolvectl status` # check systemd-resolved status
- `resolvectl query <domain>` # test resolution via systemd
- `cat /etc/resolv.conf` # view resolver configuration
- `systemd-resolve --status` # detailed resolver information
- `dig @<dns-server> <domain>` # query specific DNS server
- `dig +trace <domain>` # trace the full DNS path
- `watch -n 1 'dig <domain>'` # continuous testing
- `curl -v <url>` # test application DNS resolution

What this could mean

- Upstream DNS latency or outages
- DNS server overload
- Local resolver or caching issues
- High query volume under load
- Network/firewall issues affecting DNS (port 53)
- Short TTL or aggressive cache expiry
- Application DNS behavior (no caching, frequent lookups)

What to check

- Can you resolve the domain consistently?
- Is the issue local or affecting multiple servers?
- Are you using the expected DNS servers?
- Is systemd-resolved working correctly?
- Check `/etc/resolv.conf` for correct nameservers
- Are search domains causing delays?
- What is the TTL for the affected records?
- Is there high DNS latency or packet loss?
- Are firewall rules blocking or rate-limiting DNS traffic?
- Check upstream DNS provider status



Senior Engineer Tip

Test resolution from multiple points (localhost, another server, outside network). Compare results and latency to identify where the failure is occurring.

2 A DNS configuration change causes widespread service degradation. What do you investigate?

Key commands

- `dig <domain> +ttl` # check record TTL
- `dig <domain> any` # view all DNS records
- `host <domain>` # simple record lookup
- `resolvectl dns` # current DNS servers
- `resolvectl domain` # search domains
- `systemd-resolve --flush-caches` # clear local DNS cache
- `dig @8.8.8.8 <domain>` # test public DNS server
- `dig @<internal-dns> <domain>` # test internal DNS server
- `journalctl -u systemd-resolved` # check resolver logs
- `cat /etc/resolv.conf` # verify configuration

What this could mean

- Incorrect nameserver configuration
- Invalid or missing DNS records
- Reduced TTL causing high query load
- Propagation delays
- Misconfigured search domains
- Caching issues (old or negative cache)
- Configuration pushed to all servers (wide blast radius)

What to check

- What exactly changed in the DNS configuration?
- Are the correct records present and valid?
- Has the change propagated to all DNS servers?
- Is the TTL appropriate?
- Are clients using the expected DNS servers?
- Is there a negative cache that needs to be cleared?
- Are search domains causing unexpected lookups?
- Did the change affect multiple environments?
- Check application logs for DNS-related errors
- Verify resolution from different locations (internal/external)

3 Prevent future DNS incidents

- Use multiple, reliable DNS servers (high availability)
- Set appropriate TTL values
- Monitor DNS latency and error rates
- Implement caching at the application or system level
- Test DNS changes in a staging environment first
- Document DNS configuration and change process
- Set up alerts for DNS failures and high latency

Common symptoms

- Intermittent DNS timeouts
- Slow name resolution
- Applications failing to connect using hostnames
- Works with IP address but not hostname
- High latency during peak traffic
- Wide service impact after DNS changes
- Increased DNS query rate and upstream errors

PACKET LOSS, MTU AND NETWORK PERFORMANCE

Diagnose | Troubleshoot | Solve | Think Like a Senior Engineer

“Network performance issues are often caused by what’s invisible. Learn to find the real cause.”

1 Small requests succeed while large HTTPS requests hang. What do you investigate?

Key commands

- `ping -c 5 <destination>` # basic connectivity test
- `ping -M do -s 1472 <destination>` # test with specific packet size (no fragment)
- `tracert <destination>` # discover path MTU
- `tracert -d <destination>` # trace network path
- `curl -v https://<url>` # test HTTPS download
- `wget --debug https://<url>` # detailed HTTP debugging
- `tcpdump -i <interface>` # capture and analyze packets
- `ss -ti` # check TCP connection details
- `ip link show` # check interface status
- `ethtool <interface>` # check interface statistics and features
- `mtr <destination>` # real-time network path analysis

What this could mean

- MTU mismatch between network devices
- Packet fragmentation or dropped fragments
- Path MTU Discovery (PMTUD) blocked
- Firewalls or security devices dropping large packets
- TCP packets being silently dropped
- Network interface or driver issues
- Load balancer or proxy limitations
- Asymmetric routing or network path issues

What to check

- What is the path MTU between source and destination?
- Are ICMP packets allowed (required for PMTUD)?
- Do large packets get dropped or fragmented?
- Is there packet loss on any hop?
- Are there interface errors (TX/RX drops)?
- Is the issue only affecting specific networks or destinations?
- Does the problem occur with IPv4, IPv6, or both?
- Are firewalls, load balancers, or proxies in the path?
- Check TCP retransmissions and timeouts.
- Compare behavior with different packet sizes.



Senior Engineer Tip

If small packets work but large packets fail, always suspect an MTU or fragmentation issue. Test with increasing packet sizes to find the exact threshold.

2 Network throughput suddenly collapses even though bandwidth capacity appears sufficient. What do you investigate?

Key commands

- `jnet3 -c <server>` # test raw network throughput
- `ping -i 0.2 <destination>` # continuous ping to check packet loss
- `mtr -rw <destination>` # real-time path and loss analysis
- `ss -s` # socket statistics
- `netstat -s` # network protocol statistics
- `ethtool -S <interface>` # detailed interface error counters
- `tcpdump -i <interface>` # capture packets for analysis
- `sar -n DEV 1` # real-time interface statistics
- `nstat` # detailed TCP/UDP/ICMP counters
- `tracert <destination>` # check path MTU and route

What this could mean

- High packet loss causing retransmissions
- Network device or interface errors
- MTU or fragmentation issues
- Congestion or buffer overflows
- QoS or rate limiting
- Hardware problems (NIC, switches, cables)
- CPU saturation on network device
- DDoS or unusual traffic patterns

What to check

- What is the current packet loss percentage?
- Are there high retransmissions or TCP timeouts?
- Do interface counters show errors, drops, or collisions?
- Is the MTU consistent across the entire path?
- Is there congestion on the network or firewall?
- Are there QoS policies or rate limits?
- Does the issue affect all traffic or only specific destinations?
- Check CPU and memory usage on network devices.
- Compare performance at different times (peak vs off-peak).

3 Prevent future network performance issues

- Use consistent MTU across all network devices
- Monitor packet loss, latency, and retransmissions
- Enable and verify Path MTU Discovery
- Check interface health and error counters regularly
- Set up network monitoring and alerts (e.g. Prometheus, Zabbix)
- Document network topology and MTU settings
- Test changes in a staging environment

Common symptoms

- Large file downloads or HTTPS requests hang
- Intermittent connectivity or slow performance
- High packet loss or retransmissions
- Interface error counters increasing
- Network throughput lower than expected
- Applications timing out during high traffic
- Works on some networks but fails on others

LINUX PERMISSIONS AND IDENTITY FAILURES

Diagnose | Troubleshoot | Solve | Think Like a Senior Engineer

"Permissions problems are not just about file modes. Understand the full identity and access model."

1 An application suddenly receives "Permission denied" even though Unix permissions look correct.

What do you investigate?

Key commands

- `ls -l <file>` # check permissions and owner
- `id` # check current user and groups
- `stat <file>` # detailed file metadata
- `getfacl <file>` # view ACLs
- `setfacl -m u:<user>:rwx <file>` # modify ACLs
- `namei -l <path>` # check permissions on each path component
- `ps -ef | grep <app>` # check which user the app is running as
- `sudo -u <user> whoami` # test as the application user
- `dmesg | grep -i denied` # check kernel denial messages
- `ausearch -m avc -ts recent` # check recent SELinux denials
- `cat /etc/selinux/config` # view SELinux mode
- `aa-status` # check AppArmor status

What this could mean

- Ownership or group mismatch (UID/GID)
- Missing ACLs or incorrect ACL entries
- SELinux or AppArmor blocking access
- Mount options (read-only, noexec, nosuid)
- Capabilities or privilege requirements
- Application running as a different user
- Path or symlink permission issues
- File system on a different security context

What to check

- Are you running as the expected user and group?
- Does the file or directory have the correct ownership?
- Are there ACLs that override normal permissions?
- Is SELinux or AppArmor blocking access?
- Are the mount options restricting operations?
- Does the application need additional capabilities?
- Are there symlinks with different permissions?
- Is the file system on an NFS or external mount with root-squash or permission restrictions?
- Check audit logs for denial messages.
- Verify the full path and all parent directories.



Senior Engineer Tip

Always verify the runtime identity of the application (process user, groups, capabilities and security context). Do not assume it runs as root or the same user you used for testing.

2 A deployment changes ownership recursively and breaks multiple services.

What do you investigate?

Key commands

- `ls -lR <directory>` # recursive permission check
- `find <path> -user <user>` # find files by owner
- `find <path> -group <group>` # find files by group
- `getfacl -R <directory>` # check recursive ACLs
- `stat <directory>` # check directory permissions
- `systemctl status <service>` # verify service status
- `journalctl -u <service>` # check service logs

What this could mean

- Incorrect recursive chown or chmod
- Lost group ownership or setgid bits
- Permissions changed on shared directories
- Services unable to read/write required files
- Incorrect ownership on configuration or data directories
- Deployment script running with wrong user
- Impact on multiple services due to shared paths

What to check

- Are the correct owners and groups in place?
- Are shared directories still accessible to all services?
- Were setgid bits removed (affects new files)?
- Do services have the expected read/write permissions?
- Are there files that should not have been changed?
- Check deployment scripts for recursive changes.
- Review recent changes with audit logs or version control.

3 A privileged operation works for root but fails for the service account.

What do you investigate?

Prevention strategies

- Follow the principle of least privilege.
- Use specific capabilities instead of running as root.
- Document ownership and permission requirements.
- Test deployments in a staging environment.
- Use configuration management to enforce consistent permissions.
- Monitor audit logs for permission denials.
- Review mount options and security policies regularly.

What this could mean

- Missing Linux capabilities (e.g., CAP_NET_BIND_SERVICE)
- Operation requires root privileges
- File or directory not accessible to the service account
- SELinux/AppArmor policy restrictions
- Mount options like nosuid or noexec
- Environment or PATH differences
- Service account does not have sufficient group membership

Common error messages

- Permission denied
- Operation not permitted
- Failed to open file
- cannot execute binary file
- SELinux: access denied
- AppArmor: operation not permitted

KERNEL AND RESOURCE LIMIT FAILURES

Diagnose | Troubleshoot | Solve | Think Like a Senior Engineer

“Understand Linux kernel limits and resource controls before they take your services down.”

1 A high-traffic server fails to create additional processes or threads.

What do you investigate?

What this could mean

- Process limit reached (PIDs)
- User or system ulimit reached
- Kernel pid_max limit hit
- cgroup process limit (pids.max)
- Out of kernel memory (task_struct)
- Too many threads per process
- Container or namespace limit reached

Key commands

- `ulimit -a` # view current user limits
- `ulimit -u` # show max user processes
- `cat /proc/sys/kernel/pid_max` # check maximum PIDs
- `cat /proc/sys/kernel/threads-max` # check maximum threads
- `ps -elf | wc -l` # count total threads
- `ps -u <user> | wc -l` # count user processes
- `cat /proc/<pid>/limits` # check process limits
- `cat /proc/sys/vm/max_map_count` # check memory map areas
- `dmesg | tail -n 50` # check kernel messages
- `journalctl -k | tail -n 50` # view kernel logs

What to check

- Have you reached the PID limit or user process limit?
- Is pid_max or threads-max too low?
- Are there zombie processes consuming PIDs?
- Are cgroup limits restricting process creation?
- Is the application creating too many threads?
- Are you running inside a container with lower limits?
- Are there kernel errors in dmesg or journal?
- Is kernel memory exhausted?
- Are limits consistent across reboots?



Senior Engineer Tip

Always check system, user, and cgroup limits together. A process may fail to start even when CPU and RAM look fine. Limits are often the real bottleneck.

2 The application fails under load despite significant CPU and RAM remaining.

What do you investigate?

What this could mean

- File descriptor exhaustion
- Process or thread limit reached
- Memory map limit reached (vm.max_map_count)
- Kernel resource limits (inotify, epoll, message queues)
- cgroup CPU or memory throttling
- I/O limits or device queues
- Application hitting internal limits

Key commands

- `cat /proc/<pid>/limits` # check process limits
- `cat /proc/sys/fs/file-max` # check system file limits
- `cat /proc/sys/fs/inotify/max_user_watches` # check inotify limit
- `cat /proc/sys/net/core/somaxconn` # check socket backlog limit
- `cat /sys/fs/cgroup/<group>/` # check cgroup limits
- `top -H -p <pid>` # view threads and CPU usage
- `ps -elf | grep <process>` # list process threads
- `ss -s` # socket statistics
- `cat /proc/meminfo` # check memory information
- `dmesg | grep -i limit` # look for limit errors in logs

What to check

- Are there exhausted file descriptors or inotify watches?
- Is the socket backlog (somaxconn) too low?
- Are cgroup limits throttling CPU or memory?
- Is the application creating too many threads or connections?
- Are there kernel messages about resource limits?
- Is the application leaking resources over time?
- Are there per-namespace limits in containers?
- Does the issue only happen under high load?

3 Prevent future kernel and resource limit failures.

What do you implement?

Best practices

- Set appropriate ulimit values for applications
- Tune kernel parameters using sysctl
- Use cgroups to enforce and monitor resource limits
- Monitor PIDs, threads, file descriptors and memory usage
- Implement alerting before limits are reached
- Regularly review application behavior under load
- Document and version control all limit configurations

Key kernel and sysctl settings

- `kernel.pid_max`
- `kernel.threads-max`
- `fs.file-max`
- `fs.inotify.max_user_watches`
- `net.core.somaxconn`
- `vm.max_map_count`

Monitoring metrics

- Process count and thread count
- File descriptor usage
- Cgroup limits and usage
- Kernel error messages (dmesg)
- Application logs
- System resource usage trends
- Alert on limit thresholds

CONTAINERS AND LINUX INTERNALS

Diagnose | Troubleshoot | Solve | Think Like a Senior Engineer

“Understand how containers use Linux kernel features and why container behavior can be different from the host.”

1 A container is OOMKilled even though the host has several gigabytes of free memory.
What do you investigate?

Key commands

- `docker stats` # check container resource usage
- `docker inspect <container>` # view memory and CPU limits
- `cat /sys/fs/cgroup/memory/*` # check cgroup memory limits (v1)
- `cat /sys/fs/cgroup/memory.max` # check cgroup memory limit (v2)
- `dmesg | grep -i oom` # check OOM killer logs
- `journalctl -k | grep -i oom` # view kernel OOM messages
- `free -h` # check host memory usage
- `top / htop` # check host processes
- `kubectl describe pod <pod>` # check Kubernetes limits and events

What this could mean

- Container memory limit is lower than host memory
- cgroup memory limit reached
- Application memory leak
- High memory usage in page cache or tmpfs
- Kernel OOM killer terminated the container
- Misconfigured memory requests and limits (Kubernetes)
- Shared memory (/dev/shm) too small
- Host has memory, but container is restricted

What to check

- What is the container memory limit?
- Is the application using more memory than the limit?
- Check for memory leaks or high cache usage.
- Look at cgroup memory events and OOM logs.
- Is /dev/shm size sufficient?
- Are Kubernetes requests and limits configured correctly?
- Is the container running multiple processes?
- Compare container memory usage with host memory.



Senior Engineer Tip

A container is OOMKilled when it exceeds its cgroup memory limit, not the host's total memory. Always check container limits and memory usage inside the container.

2 A container reports high CPU throttling while host CPU utilization remains moderate.
What do you investigate?

Key commands

- `docker stats` # check container CPU usage
- `cat /sys/fs/cgroup/cpu/*` # check cgroup CPU limits (v1)
- `cat /sys/fs/cgroup/cpu.max` # check cgroup CPU limit (v2)
- `cat /sys/fs/cgroup/cpu.stat` # check throttling statistics
- `top / htop` # check host CPU usage
- `vmstat 1` # check CPU run queue and context switches
- `kubectl describe pod <pod>` # check CPU requests and limits

What this could mean

- CPU quota or limit is too low
- cgroup CPU throttling due to CFS (Completely Fair Scheduler)
- Noisy neighbor (resource contention)
- Application is CPU intensive
- Misconfigured Kubernetes CPU limits
- High context switching or CPU steal time
- Host has CPU, but container is restricted

What to check

- What are the container CPU limits and requests?
- Are there high throttling counts in cgroup stats?
- Is the application hitting its CPU quota?
- Check for noisy neighbors on the same host.
- Compare container CPU usage with host CPU usage.
- Is the workload multi-threaded and hitting CPU limits?
- Review Kubernetes resource configuration.

3 A containerized process behaves differently from the same process running directly on the host.
What do you investigate?

Key commands

- `docker inspect <container>` # view namespaces and settings
- `capsh --print` # check Linux capabilities
- `cat /proc/1/status` # check process capabilities inside container
- `mount | grep <path>` # check mount options
- `env | sort` # compare environment variables
- `ls -l /dev` # check device access
- `cat /proc/self/mountinfo` # view mounted filesystems
- `apparmor_status` # check AppArmor (if enabled)
- `sestatus` # check SELinux status

What this could mean

- Different namespaces (PID, network, mount, IPC, UTS, user)
- Missing Linux capabilities
- Restricted filesystem or read-only mounts
- Different environment variables or configuration
- SELinux or AppArmor restrictions
- Different kernel view inside the container
- Required devices or kernel features not available

What to check

- What namespaces are isolated in the container?
- Does the container have the required capabilities?
- Are filesystem paths mounted with expected options?
- Is the container running with a different user or UID/GID?
- Are SELinux or AppArmor policies blocking the operation?
- Are necessary devices or kernel modules available?
- Compare environment variables and configuration files with the host.
- Check if the application relies on host-specific behavior.

LOGGING FAILURE AND OBSERVABILITY

Diagnose | Troubleshoot | Solve | Think Like a Senior Engineer

“Good logs turn incidents into solutions. Understanding Linux logging is a core production skill.”

1 A production service fails, but its expected logs are missing.

What do you investigate?

Key commands

- `journalctl -u <service>` # view service logs
- `journalctl -xe` # detailed logs with errors
- `journalctl --since "1 hour ago"` # logs from a specific time
- `systemctl cat <service>` # check service configuration
- `cat /etc/systemd/journald.conf` # view journald configuration
- `ls -l /var/log/` # list log files
- `find /var/log -name "*.log"` # find log files
- `grep -r "ERROR" /var/log` # search for errors
- `docker logs <container>` # view container logs
- `kubectl logs <pod>` # view Kubernetes pod logs

What this could mean

- Logs are not being written to the expected location
- journald is not persistent (logs lost after reboot)
- Incorrect log level or logging configuration
- Application writing to a different file or stdout/stderr
- Log rotation removed old logs
- Service running in a different environment or user
- Permission issues preventing log writes

What to check

- Is the service writing to stdout/stderr or a file?
- Is journald configured to store logs persistently?
- Are logs being rotated or deleted too quickly?
- Does the service have write permissions?
- Is the correct log level configured?
- Are logs going to a different location (e.g. /var/log/app/)?
- If containerized, are logs accessible through the runtime?
- Check systemd unit file for StandardOutput and StandardError.
- Verify if logs exist but are compressed or archived.



Senior Engineer Tip

Always know where your application writes logs (stdout, file, journald, or external system). During an incident, confirm the log path, permissions, and rotation settings first.

2 journald consumes excessive disk space during an incident.

What do you investigate?

Key commands

- `journalctl --disk-usage` # check disk usage
- `journalctl --vacuum-size=1G` # cleanup logs by size
- `journalctl --vacuum-time=7d` # cleanup logs by time
- `cat /etc/systemd/journald.conf` # view journald configuration
- `du -sh /var/log/journal/` # check journal directory size
- `ls -lh /var/log/` # check log directories
- `grep -i rate /etc/systemd/journald.conf` # check rate limiting settings
- `systemctl restart systemd-journald` # restart journald after changes

What this could mean

- High log volume during the incident
- No log rotation or size limits configured
- journald persistent storage filling up /var/log
- Rate limiting not enabled
- Application or system component spamming logs
- Logs not being archived or forwarded

What to check

- Is Storage set to persistent or volatile?
- Are SystemMaxUse, RuntimeMaxUse, or MaxRetentionSec set?
- Are logs being forwarded to an external system (e.g. ELK, Loki)?
- Is rate limiting (RateLimitIntervalSec, RateLimitBurst) enabled?
- Which service or component is generating high log volume?
- Are old logs being cleaned up automatically?
- Do you need to adjust log levels to reduce noise?
- Is there enough disk space protection for /var/log?

3 Implement proper logging and disk protection.

What do you implement?

Example journald configuration

```
Storage=persistent      # store logs on disk
SystemMaxUse=1G         # max disk space for logs
RuntimeMaxUse=200M      # max runtime log space
MaxRetentionSec=30day    # keep logs for 30 days
RateLimitIntervalSec=30s # rate limit interval
RateLimitBurst=1000      # max messages per interval
```

Best practices

- Configure persistent journald storage
- Set log size and retention limits
- Enable rate limiting to prevent log floods
- Use log rotation for application logs (logrotate)
- Forward logs to a centralized logging system
- Monitor disk usage and set up alerts
- Ensure applications log meaningful and structured messages
- Regularly review and clean up old logs

Monitoring and alerting

- Monitor /var/log disk usage
- Alert on high journald disk usage
- Monitor log volume spikes
- Set up log forwarding (e.g. Loki, ELK, CloudWatch)
- Review logs regularly for anomalies

PRODUCTION PERFORMANCE INVESTIGATION

Diagnose | Troubleshoot | Solve | Think Like a Senior Engineer

"Performance problems are solved with data, not guesses. Measure, compare and find the real cause."

1

- Users report latency, but infrastructure dashboards appear normal.
- A server becomes slow only during specific traffic periods.
- Performance deteriorates after an apparently harmless configuration change.

What do you investigate?

Key commands

- `top / htop` # real-time system overview
- `vmstat 1` # CPU, memory, I/O, context switches
- `sar -u 1 10` # detailed CPU statistics
- `sar -r 1 10` # memory usage and paging
- `sar -d 1 10` # disk I/O and latency
- `sar -n DEV 1 10` # network interface statistics
- `pidstat -u 1` # per-process CPU usage
- `pidstat -r 1` # per-process memory usage
- `pidstat -d 1` # per-process I/O usage
- `iostat -xz 1` # detailed disk performance
- `perf top` # see where CPU is being used
- `perf record -g` # record performance data
- `ss -s` # socket and connection stats
- `dmesg | tail -n 50` # check for kernel issues

What this could mean

- The issue is at the application layer, not infrastructure
- Short, high-traffic periods are causing resource saturation
- A recent change introduced new bottlenecks
- High latency due to CPU, memory, disk or network saturation
- Lock contention, inefficient queries or increased I/O
- Thread or connection exhaustion
- Caching issues or missing cache
- Downstream service slowness (database, API, DNS, etc.)
- Performance regression after configuration change
- Monitoring is missing key metrics or has wrong thresholds

What to check (USE method)

- CPU: Is it highly utilized? Any CPU steal or iowait?
- Memory: Any swapping? High page faults?
- Disk: High I/O wait? High latency? Disk saturation?
- Network: High throughput, errors or retransmissions?
- Use USE method: Utilization, Saturation, Errors
- Check latency distributions (p50, p95, p99)
- Compare current metrics with historical baselines
- Look for correlation with traffic, deployments or changes
- Identify top resource-consuming processes
- Check application logs for slow operations
- Validate downstream dependencies (DB, APIs, DNS, etc.)
- Review recent configuration changes



Senior Engineer Tip

Always establish baselines. A system is only "normal" when compared to its historical behavior. Correlate metrics across CPU, memory, disk, network and application logs.

2

- A server becomes slow only during specific traffic periods.
- What do you investigate?**

Key commands

- `sar -u 1 60` # CPU usage over time
- `sar -r 1 60` # memory usage over time
- `sar -d 1 60` # disk I/O over time
- `sar -n DEV 1 60` # network usage over time
- `pidstat 1` # process activity over time
- `vmstat 1` # system summary over time
- `watch -n 1 'ss -s'` # watch socket stats
- `journalctl -xe --since "1 hour ago"` # check for error patterns

What this could mean

- Traffic spikes are causing resource saturation
- Auto-scaling or load balancing is not effective
- Database or external service becomes a bottleneck
- Connection pools or thread pools are exhausted
- Caching is not working or is being bypassed
- I/O contention during peak hours
- A cron job or batch process is consuming resources
- Duty cycle issues (e.g. backups, log rotation, reporting)

What to check

- Compare metrics during peak vs. normal periods
- Identify which resource is hitting saturation first
- Check for scheduled jobs or background tasks
- Analyze application logs for slow requests
- Validate external dependencies and their latency
- Ensure alerts are configured for peak conditions

3

- Performance deteriorates after an apparently harmless change.
- What do you investigate?**

Recent change analysis

- Check deployment history (git, CI/CD, configuration management)
- Review system and application logs around the change time
- Compare configuration differences
- Validate if new features, parameters or dependencies were introduced
- Use baselines to confirm when the issue started

Common performance symptoms

- High response time or latency
- Increased CPU, memory or disk I/O
- Higher error rates or timeouts
- Slow database queries
- Increased network latency
- Service becomes unresponsive under load
- Degradation only during specific time periods

SECURITY AND HOST COMPROMISE INVESTIGATION

Diagnose | Troubleshoot | Solve | Think Like a Senior Engineer

"Detect, investigate and contain security incidents without destroying forensic evidence."

1

- An unknown process begins making outbound network connections.
- CPU usage suddenly spikes and an unfamiliar binary appears in /tmp.

What do you investigate?

Key commands

- | | |
|-------------------------------|---|
| • ps auxf | # view process tree (ancestry) |
| • pstree -ap | # show parent/child processes |
| • ss -tulnp | # check listening and established sockets |
| • lsof -i | # list open network connections |
| • netstat -tulnp | # network connections (older systems) |
| • top / htop | # real-time process monitoring |
| • pidstat -p ALL 1 | # per-process CPU and memory usage |
| • find /tmp -type f -mtime -1 | # find recent files in /tmp |
| • ls -la /tmp | # check suspicious binaries |
| • file <binary> | # identify file type |
| • strings <binary> | # look for suspicious strings |
| • sha256sum <binary> | # generate file hash for analysis |

What this could mean

- Malware, reverse shell or C2 (command and control)
- Compromised user account
- Cryptomining or unauthorized workload
- Persistence mechanism has been installed
- Privilege escalation attempt
- Data exfiltration
- Suspicious activity by a legit process (process hijacking)
- Lateral movement to other systems
- Insider threat

What to check

- What process started the suspicious process?
- Is the outbound connection expected?
- Which user is running the process?
- Are there new or modified files in /tmp, /var, or home dirs?
- Check authentication logs (/var/log/auth.log)
- Look for unusual sudo or privilege escalation activity
- Review cron jobs and systemd units for persistence
- Check for new users or modified SSH keys
- Look for modified binaries or library files
- Verify file integrity (hash comparison)
- Identify indicators of lateral movement
- Collect evidence before making major changes



Senior Engineer Tip

Contain the threat, but do not destroy evidence. Capture memory, logs and disk artifacts before removing files or killing processes. Document everything you do.

2

- CPU usage suddenly spikes and an unfamiliar binary appears in /tmp.

What do you investigate?

Key commands

- | | |
|--|-------------------------------------|
| • crontab -l | # check user cron jobs |
| • ls -la /etc/cron.* | # check system cron jobs |
| • systemctl list-units --type=service | # list systemd services |
| • systemctl list-timers | # list systemd timers |
| • cat /etc/passwd | # check for new users |
| • sudo -l | # review sudo permissions |
| • find / -type f -perm -4000 2>/dev/null | # find SUID binaries |
| • grep -r "" | # search for suspicious entries |
| • rpm -Va 2>/dev/null | # verify package integrity (RPM) |
| • debsums -s 2>/dev/null | # verify package integrity (Debian) |

Common persistence mechanisms

- Cron jobs (user and system)
- Systemd services and timers
- Shell profile modifications (.bashrc, .profile, etc.)
- SSH authorized_keys
- SUID/SGID binaries
- Kernel modules
- Modified system binaries or libraries
- Web shells (on servers running web applications)
- Container escape or backdoors

What to check

- Are there unexpected cron jobs or systemd units?
- Have new users or SSH keys been added?
- Are any files modified recently in critical directories?
- Are there suspicious SUID/SGID files?
- Check web server directories for unexpected files
- Look for unusual network connections or data transfers
- Review application logs for suspicious activity
- Check container environments if applicable
- Validate file hashes against a known good baseline

3

- Contain the incident and preserve evidence for further analysis.

What steps do you take?

Evidence to preserve

- Copy suspicious binaries (do not modify)
- Collect logs (/var/log, journalctl)
- Capture memory if possible (for volatile evidence)
- Record network connections and process list
- Document system state, commands and timestamps

Containment strategies

- Isolate the host from the network (if required)
- Block malicious outbound connections (firewall)
- Disable compromised user accounts
- Stop malicious processes (after capturing evidence)
- Remove persistence mechanisms
- Rotate credentials and keys
- Apply patches and close vulnerabilities
- Monitor for continued malicious activity

CASCADING PRODUCTION FAILURE

Diagnose | Troubleshoot | Solve | Think Like a Senior Engineer

"A small failure can become a big outage. Understand dependencies, control the blast radius."

1 The incident

- One Linux host begins failing and several dependent services degrade.
- Retry traffic turns a small dependency failure into widespread saturation.

What do you investigate?

What this could mean

- A single host or service is a critical dependency
- Cascade of retries increases load on other services
- Connection exhaustion across multiple services
- Queues grow and delays propagate
- Lack of timeouts, circuit breakers or load shedding
- Shared infrastructure (DB, cache, DNS, API, message queue)
- Poor isolation and large blast radius

Key commands

- | | |
|----------------------------|---------------------------------------|
| systemctl status <service> | # check service status |
| journalctl -u <service> | # view service logs |
| ss -s | # socket and connection summary |
| ss -tanp | # view active connections |
| netstat -s | # network statistics |
| top / htop | # real-time resource usage |
| sar -u 1 10 | # CPU usage over time |
| sar -n DEV 1 10 | # network usage over time |
| iostat -xz 1 10 | # disk I/O and latency |
| docker ps / docker stats | # container status and resource usage |
| kubectl get pods | # pod status (Kubernetes) |
| kubectl top pods | # pod resource usage |
| dig / nslookup | # check DNS dependencies |
| curl -I <url> | # test dependent endpoints |
| redis-cli info | # check cache health (e.g. Redis) |
| mysqladmin status | # check database health |

What to check

- Which services are failing first?
- What are the direct and indirect dependencies?
- Are retries, timeouts and circuit breakers configured?
- Are queues growing (message queues, job queues)?
- Are there connection exhaustion or high error rates?
- Is the issue isolated or spreading across services?
- Are caches, databases or shared services affected?
- Check logs for errors, timeouts and retries.
- Compare metrics before and after the incident.
- Identify the blast radius (which services and users).
- Look for recent changes or deployments.



Senior Engineer Tip

Mitigate the blast radius early. You do not need to fully find the root cause before taking action. Stabilize the system, protect dependent services and buy time.

2 Retry traffic and dependency failures

- Retry logic turns a small failure into widespread saturation.
- Understand and control retries.

What do you investigate?

Common causes

- Aggressive retry logic without backoff
- Missing or incorrect timeouts
- No circuit breakers or bulkheads
- Synchronous calls to slow services
- Connection pool exhaustion
- Queue buildup and resource contention
- Cascading failures across microservices
- External dependency (DB, API, DNS, third party)

3 Implement mitigation and resilience

- Reduce impact, protect users and keep critical services running.

What steps do you take?

Dependency mapping example



Mitigation strategies

- Implement or tune timeouts and retry policies
- Use circuit breakers and bulkheads
- Apply load shedding and rate limiting
- Scale critical services or add replicas
- Isolate failing components
- Use fallback responses and graceful degradation
- Drain traffic from unhealthy instances
- Communicate with stakeholders and monitor closely

Key takeaways

- Understand your system's dependencies.
- Design for failure, not just success.
- Control retries, timeouts and connection limits.
- Limit the blast radius with isolation and resilience patterns.
- Mitigate early, even before root-cause completion.
- Always communicate and monitor the recovery.

STAFF-LEVEL LINUX INCIDENT JUDGMENT

Think Critically | Make the Right Trade-offs | Lead with Experience

"It's not just what you do, but why you do it. Senior engineers make informed decisions."

1 Production is failing and rebooting the host would probably restore service.

Do you reboot immediately?

Investigation commands

- uptime # system load and uptime
- dmesg | tail -n 50 # recent kernel messages
- journalctl -xe # recent errors
- systemctl status <service> # check service status
- ps aux --sort=-%cpu | head # top resource consumers
- lsof | head # open files
- netstat -tulnp # network connections
- sar -n 1 5 # quick performance snapshot
- vmstat 1 5 # system resource summary

What to consider

- Current impact and severity (SLO / user impact)
- Is the system still partially functional?
- Can you capture critical evidence before rebooting?
- Will reboot cause data loss or longer downtime?
- Are there alternative mitigations (restart service, isolate)?
- What is the rollback plan if the issue persists?
- Communication with stakeholders

Decision framework

- Assess business impact and urgency
- Capture logs and diagnostics if possible
- Try lower-impact mitigations first (restart service, kill process)
- Reboot if the impact is severe and system is unstable
- Document the decision and reasoning
- Validate service recovery after reboot
- Follow up with root-cause analysis



Senior Engineer Tip

If you must reboot, collect key evidence first (logs, dmesg, process list, network connections). A fast reboot should not erase your ability to learn from the incident.

2 You discover a kernel tuning change that improves throughput but reduces failure tolerance.

Would you deploy it?

Relevant commands / examples

- sysctl -a | grep <parameter> # check current kernel parameters
- sysctl -w <parameter>=<value> # test parameter change (temporary)
- cat /proc/sys/<parameter> # view current value
- sysctl -p # reload configuration
- uname -r # kernel version
- cat /etc/sysctl.conf # persistent configuration
- dmesg | grep <keyword> # check for related kernel messages

What to evaluate

- Performance gain versus increased risk
- Impact on stability, reliability and failure tolerance
- Does it meet business and SLO requirements?
- Have you tested it in a non-production environment?
- Is there a rollback plan?
- Long-term operational complexity
- Team knowledge and support

Decision framework

- Test in staging or a canary environment first
- Measure performance improvement and side effects
- Evaluate against SLOs and risk tolerance
- Get peer review and change management approval
- Ensure a clear rollback plan
- Monitor closely after deployment
- Document the change and results

3 A recurring Linux incident can be solved by adding capacity or by redesigning the application.

How do you decide?

Decision process

- Analyze root cause and frequency of the incident
- Compare short-term mitigation (add capacity) vs long-term fix (redesign)
- Consider total cost of ownership (infrastructure, people, maintenance)
- Evaluate risk, scalability and operational complexity
- Get stakeholder input (engineering, product, finance)
- Choose the option that meets business goals and reduces future risk

Factors to consider

- Is this a short-term incident or a recurring problem?
- Will adding capacity only delay the problem?
- What is the root cause (infrastructure, application, design)?
- Cost comparison (quick fix vs long-term solution)
- Operational complexity and maintenance effort
- Impact on reliability, scalability and future growth
- Time to implement and business priorities

Key takeaways for staff-level engineers

- Balance speed, risk and long-term stability.
- Make decisions based on data, not assumptions.
- Consider the bigger picture (SLOs, cost, complexity, team impact).
- Always have a rollback plan.
- Communicate clearly with stakeholders.
- Focus on prevention, not just reaction.
- Document your decisions and lessons learned.